
Evaluating the Role of Domain Knowledge in AlphaZero-Style Go Agents

Nitzan Saar

Abstract

This work presents the implementation and training of an AlphaZero-style reinforcement learning agent for the game of Go using consumer-grade hardware. The study investigates the role of domain knowledge by comparing an agent trained without handcrafted features or domain-specific enhancements to existing open-source models, such as KataGo, that incorporate domain knowledge.

Performance is evaluated throughout training by measuring win rates against a set of fixed-strength KataGo benchmarks. The results show that the model achieves its highest performance early in training, approximately after 24 hours, but does not exhibit sustained improvement. Instead, performance degrades or fluctuates at low levels over subsequent training iterations, indicating instability in the learning process. Furthermore, the agent fails to achieve meaningful performance against stronger opponents, suggesting limited scalability without domain-specific enhancements.

Overall, these findings suggest that domain-specific enhancements may contribute to improved performance, greater training stability, and more effective scaling in reinforcement learning systems.

1. Introduction

The game of Go is widely regarded as one of the most complex board games due to its vast search space and strategic depth, making it a longstanding benchmark for artificial intelligence research. In 2017, DeepMind introduced AlphaGo Zero, a reinforcement learning system that achieved superhuman performance through self-play without relying on handcrafted features or domain-specific heuristics. Subsequent work introduced AlphaZero, a more general reinforcement learning framework capable of learning multiple board games using the same approach. Later systems, such as KataGo, demonstrated that incorporating domain knowledge and algorithmic improvements can significantly improve both training efficiency and overall performance.

This work investigates the role of domain knowledge by implementing an AlphaZero-style Go agent and evaluating its performance against domain-enhanced models. The primary objective is to examine how the absence of domain-specific enhancements affects training efficiency, stability, and overall playing strength under constrained computational resources.

2. Methods

2.1. Self-play training loop

The training process follows an iterative self-play reinforcement learning loop. At a high level, the agent repeatedly alternates between generating training data through self-play and updating the neural network using this data.

At each iteration, the current neural network is used to play games against itself. During these games, moves are selected using Monte Carlo Tree Search (MCTS) guided by the network’s policy and value predictions. For each move, the corresponding game state s , the improved policy π derived from MCTS visit counts, and the final game outcome z are recorded. These tuples (s, π, z) are stored in a replay buffer.

After a sufficient number of self-play games are generated, the neural network is updated by sampling mini-batches from the replay buffer. The network parameters are optimized with stochastic gradient descent to match the MCTS-improved policy and predict the final game outcome. The updated network is then used to generate new self-play games, and the process repeats.

This iterative loop allows the agent to continually improve by learning from increasingly strong self-play data, without relying on external datasets.

2.2. Monte Carlo Tree Search

During self-play, rather than solely relying on the output from the neural network to select the next move, 1600 iterations of Neural Network Guided Monte Carlo Tree Search (MCTS) are performed at each turn to produce a more informed policy π .

Starting from the current game state, MCTS incrementally builds a search tree where each node represents a game

position and edges correspond to actions. Each simulation consists of three phases: selection, expansion, and backup. During the selection phase, the tree is traversed from the root to a leaf node by iteratively choosing the action that maximizes a PUCT (Predictor + Upper Confidence Bound for Trees) score.

Formally, for a state s and action a , the selection score is given by:

$$U(s, a) = -Q(s, a) + (c_{\text{puct}} \cdot P(s, a) \cdot \frac{\sqrt{N(s)}}{1+N(s, a)})$$

where $Q(s, a)$ is the average value of the child node, $P(s, a)$ is the prior probability of selecting action a as predicted by the neural network, $N(s)$ is the visit count of the parent node, and $N(s, a)$ is the visit count of the child node. The constant c_{puct} controls the balance between exploration and exploitation.

The first term, $Q(s, a)$, favors actions that have previously led to strong outcomes, while the second term encourages exploration of actions that have high prior probability but have been visited relatively few times. The exploration bonus increases with the square root of the parent’s visit count, promoting broader search as the tree grows, while it is inversely proportional to the child’s visit count, ensuring that frequently explored actions are gradually deprioritized.

When a leaf node is reached, it is expanded by querying the neural network, which outputs both a policy distribution over legal moves and a scalar value estimating the expected outcome of the game from that position. The policy is used to initialize the prior probabilities of the node’s children, while the value is used to evaluate the position.

The evaluation is then propagated back through the nodes visited during the simulation in the backup phase, where visit counts are incremented and value estimates are updated. To improve computational efficiency, simulations are performed in batches: multiple leaf nodes are collected, evaluated simultaneously by the neural network, and then updated in parallel. Additionally, virtual loss is applied during traversal to discourage multiple simulations from exploring the same path within a batch.

After all simulations are completed, the visit counts of the root’s children are normalized to form a new probability distribution which serves as the improved policy used both for move selection and as a training target. During early stages of the game, moves are sampled from this distribution to encourage exploration, while in later stages, the move with the highest visit count is selected. For each move, the corresponding board state and MCTS-derived policy are stored in a replay buffer. After the game concludes, a final outcome value z of +1, -1, or 0 is assigned to each stored state, depending on whether the game resulted in a win, loss, or draw from the perspective of the player at that state s .

The resulting (s, π, z) tuples are stored in a sliding-window replay buffer where older samples are continuously replaced as new self-play data is generated.

Neural Network Guided Monte Carlo Tree Search

Require: State s , neural network f_θ , simulations K

Ensure: Improved policy π

```

1: Initialize root node with state  $s$ 
2: for  $k = 1$  to  $K$  do
3:    $s_t \leftarrow s$ 
4:    $\mathcal{P} \leftarrow \emptyset$ 
5:   Selection
6:   while  $s_t$  is in the tree and not terminal do
7:      $a \leftarrow \arg \max_a \left[ -Q(s_t, a) + c_{\text{puct}} P(s_t, a) \frac{\sqrt{N(s_t)}}{1+N(s_t, a)} \right]$ 
8:     Append  $(s_t, a)$  to  $\mathcal{P}$ 
9:      $s_t \leftarrow T(s_t, a)$ 
10:  end while
11:  Expansion
12:  if  $s_t$  is not terminal then
13:     $(P(s_t, \cdot), v) \leftarrow f_\theta(s_t)$ 
14:    Initialize children with prior  $P(s_t, \cdot)$ 
15:  else
16:     $v \leftarrow$  game outcome at  $s_t$ 
17:  end if
18:  Backup
19:  for each  $(s_i, a_i)$  in  $\mathcal{P}$  do
20:     $N(s_i, a_i) \leftarrow N(s_i, a_i) + 1$ 
21:     $Q(s_i, a_i) \leftarrow \frac{Q(s_i, a_i)(N(s_i, a_i) - 1) + v}{N(s_i, a_i)}$ 
22:     $v \leftarrow -v$ 
23:  end for
24: end for
25:  $\pi(a | s) \propto N(s, a)$ 
26: return  $\pi$ 

```

2.3. Neural Network

During inference, the network takes as input a tensor of shape (17, 19, 19). These 17 planes represent the previous 8 board states for each player (16 total) plus one plane indicating the current player to move.

This input is passed through an initial convolutional layer with a 3x3 kernel, stride 1, padding 1, and 256 output channels, followed by batch normalization and a ReLU activation. The result is then fed through a residual tower consisting of 20 residual blocks. Each residual block contains two 3x3 convolutional layers with batch normalization; ReLU is applied after the first convolution, and a skip connection adds the input of the block to the output before a final ReLU.

The output of the residual tower is then used by two separate heads: a value head and a policy head.

The value head begins with a 1x1 convolution that reduces

the 256 channels to 1, producing a tensor of shape (1, 19, 19). This can be interpreted as a single scalar per board position summarizing the learned features. This tensor is flattened into a vector of size 361 and passed through two fully connected layers to produce a single scalar value in the range $[-1, 1]$, representing the predicted outcome of the game.

The policy head applies a 1×1 convolution that reduces the 256 channels to 2, followed by batch normalization and ReLU. The result is flattened and passed through a fully connected layer to produce a vector of size $19 \times 19 + 1$, representing a probability distribution over all possible moves, including the pass move.

2.4. Training

The network consists of a shared residual trunk with separate policy and value heads. The policy head is trained using cross-entropy loss with soft targets derived from MCTS visit counts π , while the value head is trained using mean squared error (MSE) against the final game outcome z . During training, these losses are computed independently and combined as a weighted sum, which is then backpropagated through the network. The policy and value heads receive gradients from their respective losses, while the shared trunk receives gradients from both.

Parameters are updated using mini-batch stochastic gradient descent with momentum 0.9 and weight decay 1×10^{-4} , and a learning rate of 0.01 for the default 19×19 configuration. Training is performed with a batch size of 512 and 1500 mini-batch updates per training iteration. The data loader samples training examples with replacement using a weighted distribution, where each example is selected independently and examples with higher weights have a greater probability of being included in a mini-batch.

2.5. Input Representation Comparison

The input representation used in this work follows the AlphaZero formulation, consisting of 17 feature planes encoding recent board states and the current player to move. In contrast, KataGo employs a substantially richer input representation that includes additional domain-specific features such as move history, liberty counts, ko restrictions, ladder information, and territory-related signals. A comparison of the input features used by both systems is shown in Table 1. These additional features provide structured information about the game state that may improve both learning efficiency and training stability.

3. Process

The development process began with training the system on 9×9 Go rather than 19×19 Go in order to validate the

correctness of the implementation before committing to a more computationally expensive training run. Training on a smaller board reduces the search space and allows for faster iteration, making it suitable for early-stage debugging. However, 9×9 Go lacks widely accepted benchmarking standards.

To evaluate performance during early development, publicly available KataGo models with reported Elo ratings were used as benchmarks by restricting them to play on a 9×9 board. Although these models are trained for 19×19 play, this provided a consistent method for relative performance evaluation. The 9×9 experiments, as shown in Figure 1, showed an increase in win rate over time, providing confidence in the correctness of the implementation.

Following this validation, the system was extended to the standard 19×19 board for a full training run, significantly increasing the complexity of the problem.

Those same publicly available KataGo models were used as reference points in the 19×19 evaluation games. These models include reported global step counts, which serve as a rough indicator of training scale. While step counts are not directly comparable across implementations, they provide useful context for comparing the relative scale of training between the proposed system and domain-enhanced approaches.

Several design decisions were made to remain consistent with the AlphaZero framework. A gating mechanism for model selection was not implemented. While AlphaGo Zero used gating to promote improved models, AlphaZero removed this step in favor of continuous updates. In addition, a constant learning rate was used throughout training.

The final 19×19 training run was conducted over five days on a system equipped with a single NVIDIA RTX 5090 GPU and an Intel Core Ultra 9 285K CPU (24 cores). Due to time and computational constraints, longer training runs and alternative stabilization techniques were not explored.

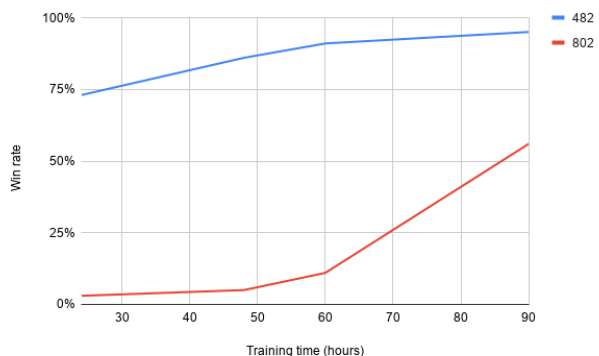


Figure 1. 9×9 Performance progression vs KataGo Benchmarks

Table 1. Comparison of input feature representations between KataGo and the AlphaZero-style agent

Channel(s)	KataGo	Channel(s)	AlphaZero-Style Agent
1	Current player’s stones	1–8	Current player’s last 8 board states
2	Opponent stones	9–16	Opponent’s last 8 board states
3–5	Liberties	17	Color to play
6	Ko ban locations	–	–
7–8	Encore states	–	–
9–13	Last 5 moves (alternating)	–	–
14–17	Ladder features	–	–
18–19	Territory count	–	–
20–21	Second encore starting stones	–	–

4. Results

Over the five day run, the AlphaZero-style agent processed approximately 112,128,000 global step samples. Model performance was evaluated at multiple checkpoints by measuring win rates against a set of fixed-strength KataGo benchmarks using 100-game matches.

Figure 2 shows the win rate of the agent across training iterations. Performance is not monotonic; the model achieves its strongest results early in training, within the first 24–48 hours, and then generally declines or fluctuates at low levels thereafter. Against the 482 Elo benchmark, performance peaks at 8% after 48 hours before steadily decreasing. Performance against the 802 Elo benchmark remains consistently low, varying between 0–3% without any clear upward trend. The 1070 Elo benchmark exhibits a sharp early spike to 18% at 24 hours, followed by a rapid drop and continued instability across later checkpoints. The agent records a 0% win rate against the 1527 Elo benchmark at all stages of training, indicating no measurable progress against stronger opponents.

A key observation is the presence of training instability. While the model is able to discover partially effective strategies early in training, it fails to retain or build upon them over time. Instead, continued training leads to inconsistent and often degraded performance, with no sustained improvement and limited generalization to higher-strength opponents.

This behavior contrasts with the 9×9 experiments, where performance improved steadily over time. The lack of similar improvement in the 19×19 setting suggests that scaling to a larger board introduces substantial additional challenges.

Several factors may contribute to this instability. The absence of domain-specific enhancements limits the quality of learning signals compared to systems such as KataGo. Additionally, the use of a constant learning rate without decay may result in large updates that overwrite previously learned behavior. Finally, the self-play training loop creates

a feedback mechanism in which the quality of training data depends on the current model, potentially amplifying errors over time.

Overall, the results indicate that under limited computational resources, an AlphaZero-style agent without domain knowledge can achieve early learning but fails to sustain improvement. Compared to KataGo models, the proposed approach demonstrates lower training efficiency and reduced stability, highlighting the importance of domain-specific enhancements in complex reinforcement learning tasks.

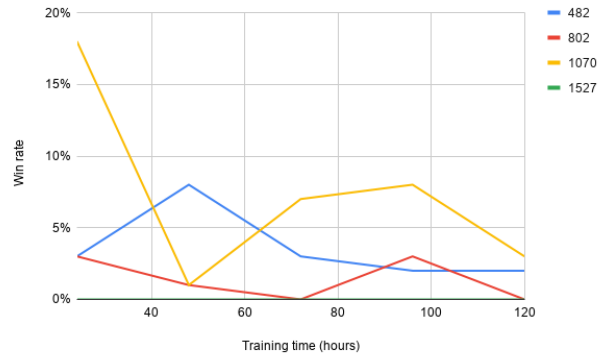


Figure 2. 19x19 Performance progression vs KataGo Benchmarks

5. Contributions

No external datasets were used; instead, all training data was generated through self-play. For evaluation and benchmarking, publicly available KataGo models were used. These models, along with their reported Elo ratings and training step information, were obtained from the KataGo training repository (<https://katagotraining.org/networks/kata1/>). These models served as fixed-strength opponents to assess the performance of the implemented system.

Table 2. Comparison of training scale between KataGo models and the AlphaZero-style agent

Type	Model / Day	Global Steps	Elo
KataGo	s1248000	1,248,000	482
KataGo	s938496	938,496	802
KataGo	s1995008	1,995,008	1070
KataGo	s4136960	4,136,960	1527
AlphaZero-style agent	Day 1	22,272,000	–
AlphaZero-style agent	Day 2	44,544,000	–
AlphaZero-style agent	Day 3	66,816,000	–
AlphaZero-style agent	Day 4	89,856,000	–
AlphaZero-style agent	Day 5	112,128,000	–

6. Citations and References

(Lightvector, 2019; Wu, 2020; Silver et al., 2017a;b)

References

Lightvector. Katago. <https://github.com/lightvector/KataGo>, 2019. GitHub repository.

Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T. P., Simonyan, K., and Hassabis, D. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *CoRR*, abs/1712.01815, 2017a. URL <http://arxiv.org/abs/1712.01815>.

Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T. P., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., and Hassabis, D. Mastering the game of go without human knowledge. *Nature*, 550:354–359, 2017b. URL <https://api.semanticscholar.org/CorpusID:205261034>.

Wu, D. J. Accelerating self-play learning in go, 2020. URL <https://arxiv.org/abs/1902.10565>.